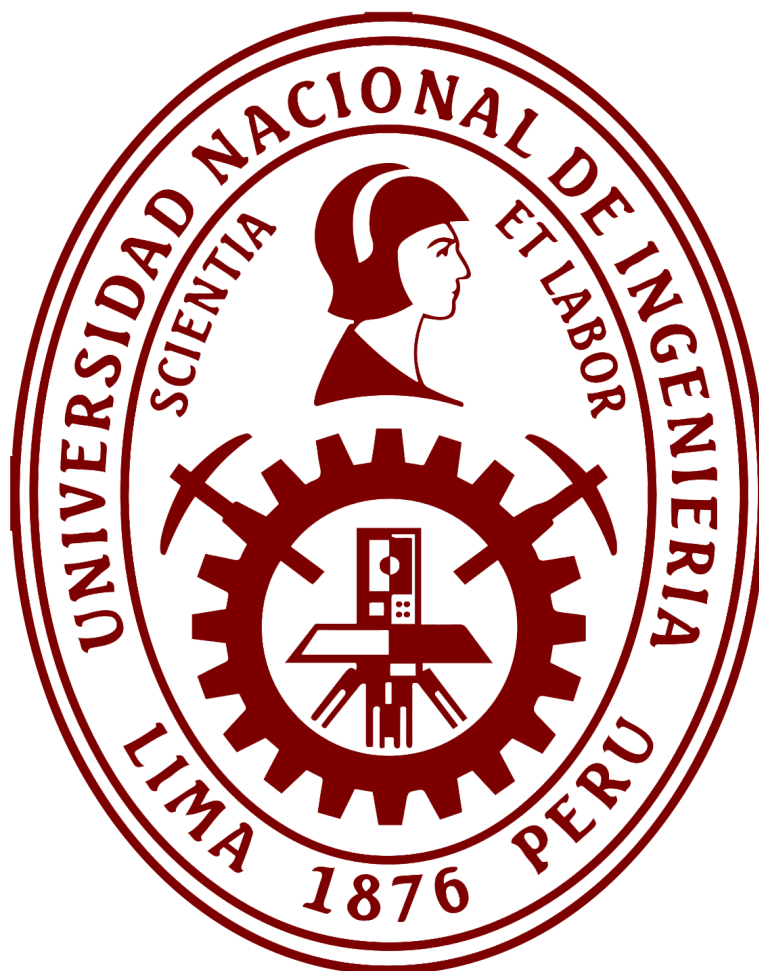


Universidad Nacional de Ingeniería
Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación



Desarrollo de Software - CC3S2A
Laboratorio, semana 1

Alumno	Código
Alvinagorta Huapaya, Angel Andres	20210047H

Docente: Quispe Torres, Manuel Alejandro

Problema 1:

- **Diseño de funciones y funcionalidad:**

- **Encapsulamiento:** La función `res_enrutamiento()` agrupa toda la lógica de lectura, procesamiento y salida de datos.
- **Punto de Entrada Seguro:** Se Implementó la condición `if __name__ == "__main__":` al final del script. Esta estructura garantiza que la función de enrutamiento sólo se ejecute cuando el archivo es invocado directamente desde la consola.

- **Estructuras de Datos Utilizadas:**

- **Listas (list):** Se utilizó una lista denominada `rutas` para almacenar las configuraciones del sistema.
- **Tuplas (tuple):** Cada ruta ingresada se almacena en la lista principal como una tupla con dos elementos: la ruta fragmentada y su contenido principal.
- **Cadenas de texto (str) y sus métodos:**
 - `strip()` : se aplica a cada línea de entrada para eliminar espacios en blanco o saltos de línea residuales, asegurando una lectura limpia.
 - `split('/')` : en lugar de comparar las URL como textos completos, las divide en segmentos independientes, permite evaluar jerarquías y detectar parámetros de forma aislada.

- **Lógica de Control y Algoritmo de Búsqueda:**

- **Filtrado por Longitud:** Para optimizar recursos, el primer filtro que aplica el sistema al comparar una transición con una ruta es verificar la cantidad de segmentos (`len(ruta_partes) == len(trans_partes)`).
- **Interacción Paralela (zip):** Se utiliza `zip()` para comparar en paralelo los segmentos de la ruta predefinida y los de la transición actual.
- **Detección de Parámetros Dinámicos:** Durante la iteración paralela, el algoritmo utiliza el método `startswith(':')` sobre el segmento de la ruta. Si resulta verdadero, el sistema comprende que no debe buscar una coincidencia exacta de texto, sino que debe capturar el valor proporcionado en la transición para añadir después del contenido, seguido de un espacio.
- **Banderas de Control:** Se Implementaron las variables booleanas `coincide` y `encontrado`. La bandera `coincide` verifica la viabilidad de una sola ruta, mientras que `encontrado` monitorea el estado global de la búsqueda. Si tras evaluar todas las configuraciones disponibles `encontrado` se mantiene como falso, se activa el mecanismo de manejo de errores, imprimiendo "404 Not Found".

Entrada:

```
3
/ HomePage
/profile ProfilePage
/user/:id UserPage (id)
4
/
/profile
/user/42
/settings
```

Salida:

```
PS C:\Users\angel\Desktop\LAB_SEM1\Problema 1> Get-Content entradaP1.txt | python Problema1.py
HomePage
ProfilePage
UserPage 42
404 Not Found
```

Problema 2:

- **Diseño de Funciones y Funcionalidad General:**
 - **Encapsulamiento:** Una función `res_banco()` agrupa toda la lógica de lectura, asociación de variables y evaluación de datos.
 - **Procesamiento de Entrada:** Se implementó la condición `if __name__ == "__main__":` al final del script para garantizar que la función principal solo se ejecute cuando el archivo es invocado directamente desde la consola.
- **Estructuras de Datos Utilizadas:**
 - **Listas (list):** Se utilizaron para almacenar la lectura masiva de datos tras emplear el método `split()` sobre `sys.stdin.read()`, facilitando el recorrido posterior por índices.
 - **Tuplas (tuple):** Se generan implícitamente al iterar sobre los diccionarios utilizando `.items()`, lo que permite extraer en cada iteración un par estructurado (`cliente, cantidad`) para ser evaluado dentro del bucle.
 - **Cadenas de texto (str):** Se utilizan al inicio como puente antes de ser procesadas, y en la etapa final mediante cadenas formateadas de forma `f"{p}{mejor_cliente}"`, asegurando que la consola imprima los espacios y el formato exacto.
 - **Tablas Hash (Diccionarios):**
 - Diccionario de Mapeo de Terminales (`terminal_a_socio`): Se estableció la estructura ID del terminal: ID del socio.
 - Diccionario Anidado (`compras_por_socio`): Se diseñó una estructura que mapea Socio -> Cliente -> Cantidad de compras.
- **Lógica de Control y Algoritmo de Evaluación:**

- **Mapeo Directo:** El programa identifica a qué socio pertenece una compra buscando directamente el terminal en el diccionario mediante `if t in terminal_a_socio`. Esto reduce el tiempo de comprobación a $O(1)$.
- **Acumulación Dinámica:** Durante la lectura de las transacciones, los contadores se actualizan dinámicamente (`compras_por_socio[p][c] += 1`). Si un cliente es nuevo, el `defaultdict(int)` lo inicializa en cero automáticamente antes de sumarle la transacción.
- **Variables de Control e Inactividad:** Antes de evaluar a los clientes de un socio particular `p`, el sistema inicializa las variables `mejor_cliente = -1` y `max_compras = -1`. Si el socio no tuvo actividad y no existe en el registro (`if p in compras_por_socio`), el bucle interno se omite e imprime automáticamente el estado por defecto `-1` de inactividad.
- **Evaluación y Desempate Manual:** Para encontrar al cliente más fiel, se itera sobre los registros de cada socio actualizando el máximo detectado (`if cantidad > max_compras`). Si se encuentra un cliente con una cantidad de compras igual al máximo actual, se aplica un desempate condicional (`elif cantidad == max_compras`), reemplazando el registro sólo si el ID del nuevo cliente es menor al que ya estaba guardado (`cliente < mejor_cliente`).

Entrada:

```
4 6 9
1 1
2 80
1 3
3 1000
2 150
4 20
1501 80
1502 150
1501 80
1501 150
1501 1
1501 3
1501 3
1502 1000
1503 1000
```

Salida:

```
PS C:\Users\angel\Desktop\LAB_SEM1\Problema 2> Get-Content entradaP2.txt | python Problema2.py
1 1501
2 1501
3 1502
4 -1
```